Reset Early, Reset Often, Eliminate Models

Martin S. Fixman[1], Fredrik Rømming[2], and Sean B. Holden[3]

University of Cambridge,
United Kingdom
[1]msf43@cl.cam.ac.uk [2]fr409@cl.cam.ac.uk [3]sbh11@cl.cam.ac.uk

Abstract

Connection-tableau provers typically construct closed tableaux by backtracking search. This gives lightweight proof procedures, but their low-memory search state leaves little derived information to reuse after a failed attempt. SATCoP addresses this by retaining grounded tableau clauses in an incremental SAT solver, allowing the prover to refute the problem once the accumulated ground clauses become propositionally unsatisfiable. We introduce SATResetCoP, which relies more directly on this persistent SAT state by resetting unproductive tableaux and using each new tableau to generate further ground clauses for the SAT solver. On TPTP, SATResetCoP improves over our leanCoP-style baseline by 54% and our SATCoP-style baseline by 18%. These results suggest that changing tableau guidance to force faster saturation of the ground SAT core improves the performance of teh prover.

1 Introduction

Connection-tableau provers prove first-order problems by constructing closed tableaux from sound tableau-construction rules[5]. Implementations usually turn this construction into backtracking search over applications of those rules. This gives a lightweight and goal-directed proof procedure, but its small search state means that failed partial tableaux leave little derived information for later attempts to reuse.

Much practical work on connection-tableau proving has therefore focused on controlling backtracking. leanCoP uses Prolog's SLD resolution for depth-first connection-tableau search with iterative deepening, and leanCoP 2.0 obtains substantial speedups by restricting this backtracking with Prolog cuts at the cost of completeness[7]. meanCoP refines this pruning strategy by separating cuts on reductions from cuts on extensions and by introducing exclusive cuts, which preserve more alternatives than the inclusive cuts used by leanCoP[4].

SATCoP addresses the same weakness from a different direction[13]. During tableau construction, it grounds encountered clauses and stores them in an incremental SAT solver. If the accumulated ground clauses become propositionally unsatisfiable, the original first-order problem can be proved before the active tableau has closed. We introduce SATResetCoP, which pushes this idea further: when the active tableau stops producing new ground clauses, it is reset, while the accumulated SAT solver state is kept. The tableau is therefore used as a generator of ground clauses, and the SAT solver becomes the persistent object from which a proof may ultimately be obtained.

We evaluate SATResetCoP on TPTP against leanCoP-style and SATCoP-style baselines implemented in the same backend. SATResetCoP solves 54% more problems than the leanCoP-style baseline and 18% more than the SATCoP-style baseline, suggesting that once ground clauses are preserved in the SAT solver, resetting unproductive tableaux can be an effective way to generate the clauses needed for a proof.

2 Preliminaries

2.1 Connection Calculus and Tableau Search

Connection tableaux and model elimination give a goal-directed alternative to the saturation style used by provers such as Vampire[14]. Instead of deriving clauses from a global clause set until a contradiction is found[1, 17], connection calculi try to close a tableau by connecting goal literals to complementary literals, either on the current path or in a fresh copy of an input clause[5, 9].

The standard clausal connection tableau has three proof-building rules[5, 10].

Start Choose an input clause and make its literals open goals.

Extension Open a branch on a clause containing a complementary literal, and add the remaining literals as new subgoals. This may require a substitution of variables with terms.

Reduction Close the current goal against a complementary literal already on the current branch. This may also require a substitution.

The rules are sound, and with fair backtracking over rule applications the calculus is complete. The calculus is not confluent: a locally sound connection can bind variables in a way that prevents later branches from closing[5]. Backtracking is therefore part of the operational search procedure, not a proof-building rule.

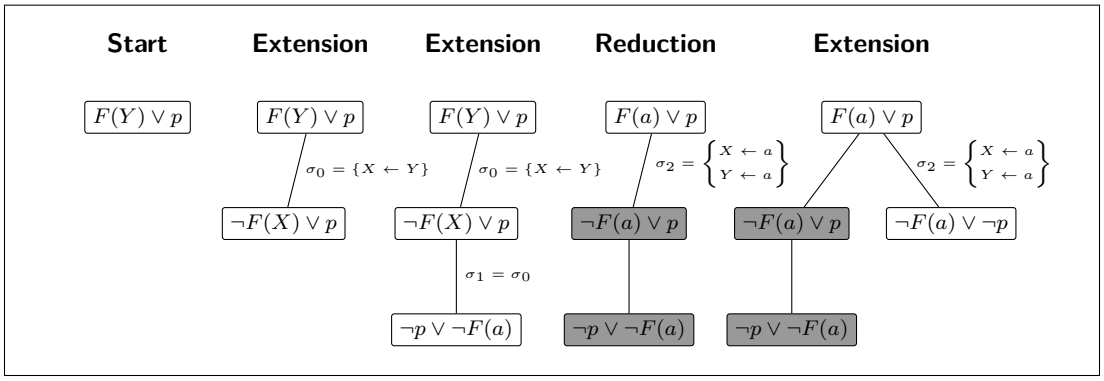


Figure 1: The three proof-building steps of a clausal connection tableau, taken from the tableau closure of SYN052-1.p[11]. The two Extension steps add an extra clause to the tableau; the Reduction step closes the nodes against a complementary literal already on the path. A full solution can, in several different strategies, be found in Appendix A.

Fredrik: I would avoid appendix, and instead have a README in the repo for code instructions

2.2 Backtracking Control

Backtracking lets a prover recover from locally sound choices that later prevent tableau closure. In leanCoP-style provers, this search is controlled mainly by iterative deepening and Prolog cuts. Iterative deepening bounds the tableau height and raises the bound when no proof is found. Cuts prune alternatives after selected reductions or extensions, reducing search but losing completeness unless a later schedule disables them[7].

meanCoP refines this pruning mechanism by separating cuts on reductions from cuts on extensions and by introducing exclusive cuts, which preserve more alternatives than the inclusive cuts used by leanCoP[4]. hopCoP takes a different route: instead of pruning alternatives directly, it learns constraints explaining why a search state is stuck, reducing repeated backtracking while preserving completeness at a fixed iterative-deepening bound[12].

2.3 SATCoP and Persistent Ground Clauses

SATCoP maintains an incremental SAT solver alongside tableau search[13]. Whenever a clause appears in the tableau, it is grounded and added to the SAT solver. If the accumulated ground clauses are propositionally unsatisfiable, proof search can stop even if the active first-order tableau has not closed.

The soundness condition is that every clause sent to the SAT solver must be a ground instance of a clause generated by the tableau. Remaining tableau variables are therefore mapped to ground terms before a clause is added. Rawson and Reger use the grounding heuristic of Schulz: choose the most frequent constant in the conjecture, falling back to a fresh constant when no conjecture constant is available[16, 13].

Later SATCoP variants also use the SAT solver to guide tableau search. Rawson and Reger describe a literal-selection refinement that prefers extension choices whose selected literal is true in the current SAT assignment[13, § 9.1].

3 Research Vehicle

3.1 SATResetCoP Specification

We propose a new tableau prover based on SATCoP, but guided by the goal of producing the ground clauses needed for a SAT refutation rather than using the leanCoP heuristics that attempt to close the current tableau.

The resulting strategy is simple: aggressively reset the tree when the SAT solver stops receiving new ground clauses. This adds a broader set of ground clauses to the solver in less time, allowing the SAT solver to reach unsatisfiability earlier.

SATResetCoP uses many of the properties of SATCoP described in the original paper and in its "Future Research" section[13, § 9.1], but changes the tableau guidance to prioritise aggressive resets.

Just like SATCoP and other provers, this approach is sound but incomplete. However, we expect the speed at which it proves unsatisfiability can increase the number of problems solved within fixed time and step limits.

Like SATCoP, the implementation keeps a persistent SAT solver alongside the active tableau search. The SAT state is not recreated during ordinary proof search. Ground clauses accumulated in earlier tableau attempts remain available in later attempts, and new clauses are added incrementally to the same solver instance.

A new virtual operation, Reset, clears the tableau, the current substitution, and the proof sequence, and starts a new tableau attempt with a specified depth bound. It does not clear the persistent SAT solver: every reset starts from an empty tree, while the SAT solver keeps the ground information produced by earlier attempts. An example of how this operation works can be seen in Figure 2.

The grounded clauses used by SATResetCoP come from clauses that have appeared in the tableau, after applying the tableau substitution accumulated so far. Start clauses are added to

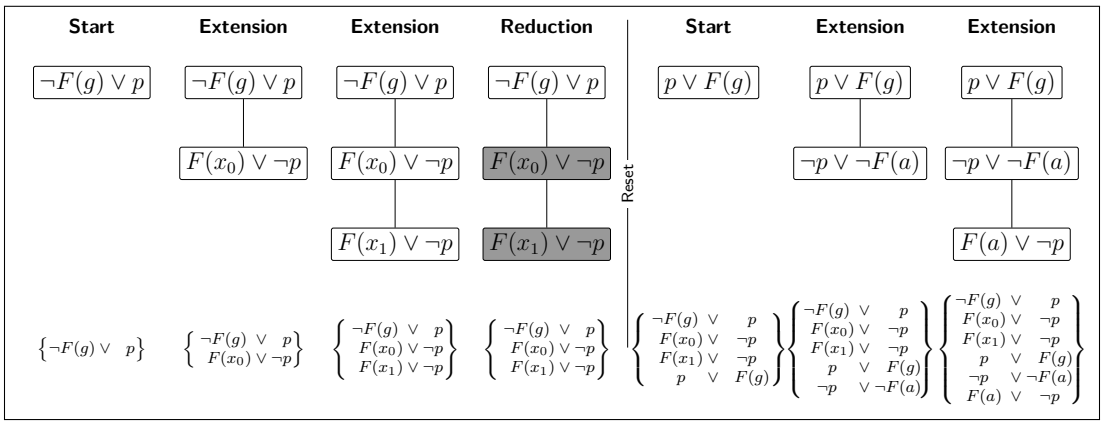


Figure 2: Example of SATResetCoP on SYN052-1.p. SATCoP and previous provers would backtrack after closing the branch with $F(x_1) \vee \neg p$; instead, SATResetCoP resets into a new tree. This accumulates the ground clauses that form the UNSAT core shown at the bottom faster than in SATCoP, allowing the SAT solver to reach unsatisfiability earlier. Substitutions are omitted for simplicity. At the end of the last extension, the accumulated ground clauses are unsatisfiable and the entire tableau prover can certify UNSAT.

the SAT state when the run is initialised. After successful Extension and Reduction steps, the implementation grounds the clauses currently present in the tableau and records the genuinely new ones, which are later flushed to the SAT solver.

The grounding scheme follows the soundness argument of Section 2.3: the SAT solver receives only ground instances of clauses generated by the tableau. Ground terms are kept as they are, and variables assigned by the current tableau substitution are replaced by their assigned terms. Only tableau variables that remain unassigned need to be replaced by a ground term. Our Python SATCoP-style variants use the simpler fallback-style choice throughout, mapping every remaining tableau variable to the same fresh artificial constant.

The reset policy does not affect the soundness of the SAT stopping condition. Every clause added to the SAT solver is a ground instance of an input clause, after applying the current tableau substitution and replacing any remaining tableau variables by the grounding constant. Therefore, if the accumulated propositional clauses are unsatisfiable, the original first-order clause set is unsatisfiable. The cuts and resets only affect which ground instances are discovered, and hence the completeness and performance of the search, not the soundness of a SAT refutation once found.

The aggressive reset-on-backtrack policy is the main implementation difference between SATResetCoP and SATCoP. At a dead end, the prover first asks whether the current tableau attempt produced any new ground clauses that have not yet been added to the SAT solver. If it did, those clauses are flushed to the SAT solver and the prover resets at the same depth. This gives the next tableau attempt the same depth bound, but a larger SAT state and a fresh randomised ordering of choices. If the attempt did not produce any new ground clauses, the prover treats the current depth as exhausted and resets at the next depth instead of spending more steps on local backtracking. The process stops as soon as the accumulated ground clauses become propositionally unsatisfiable.

SATResetCoP randomises the Start and Extension steps in the same way as SATCoP. When starting a new tree, the implementation shuffles the possible starts and the literals inside each start clause. When several extensions are possible from the leftmost open node, it shuffles the extension choices and the literal order inside the copied extension clause.

```
def solve_tableau(start_clauses):
  solver := incremental_sat_solver()
  solver.add(ground(start_clauses))
  pending := empty_set

  max_depth := 1
  tableau := empty_tableau(max_depth)

  while solve_sat(solver) != UNSAT:
    model := current_model(solver)
    actions := legal_actions(tableau)

    if any Start, Extension, Reduction in actions:
      model_rank(a) :=
        2 if a is Start
        0 if selected_literal(a) is true in model
        1 if selected_literal(a) is unknown in model
        3 if selected_literal(a) is false in model

      next_action = argmin(actions, key = model_rank)
      tableau_step(next_action)

      if next_action is Reduction or Extension:
        pending.add(ground(tableau))

    else:
      if pending is not empty:
        solver.add_all(pending)
        pending := empty_set
        tableau := empty_tableau(max_depth)
      else:
        max_depth := max_depth + 1
        tableau := empty_tableau(max_depth)
```

Listing 1: Pseudocode of the SATResetCoP control policy.

Since SATResetCoP resets the tree aggressively, it only increases the depth bound when the accumulated ground clauses have not been updated since the last Reset. This prevents trees from growing out of control, while still trying to maximise the set of accumulated ground clauses.

Pseudocode for this implementation can be found in Listing 1.

It might be useful to incorporate SATResetCoP into the initial steps of a more complete schedule; this is explored in Section 6.1.

3.2 Implementation and TPTP Benchmarking

We implement SATResetCoP and the baseline strategies leanCoP^{C}, SATCoP$_0{}^{C}$, and SATCoP^{C} using the Connections library[15], which provides primitives for implementing connection-tableau provers, together with the incremental SAT solver CaDiCaL[2]. The superscript C marks our Connections/CaDiCaL-based reimplementations of leanCoP, SATCoP$_0$, and SATCoP. Using a common backend makes the strategies easier to compare by keeping the tableau representation, rule applications, grounding interface, SAT interface, and step accounting fixed.

The source code for this implementation is available online,[1] and is released under the GNU Lesser General Public Licence[3].

For evaluation, we run SATResetCoP, leanCoP^{C}, SATCoP$_0{}^{C}$, and SATCoP^{C} on 7815 CNF problems and 8779 FOF problems from the Thousands of Problems for Theorem Provers (TPTP)

[1] https://github.com/mfixman/resetsatcop

Strategy	Explanation	Cut	SAT Refutation	SAT Guidance	Reset on Backtrack
leanCoP^C	leanCoP with initial set of predicates.	✓			
SATCoP$_0{}^C$	Based on the initial implementation of SATCoP[13].	✓	✓		
SATCoP^C	Improvements on SATCoP[13, § 9.1].	✓	✓	✓	
SATResetCoP	Novel strategy, specified in Section 3.1.	✓	✓	✓	✓

Table 1: Control features of the baseline strategies and SATResetCoP.

library[18]. These problems cover all 50 TPTP categories containing at least one CNF or FOF problem.

The library provides a varied problem set that is a standard benchmark for first-order provers. It also provides declared statuses for these problems, which are a useful sanity check for false unsatisfiability results.

4 Experimental Evaluation

4.1 Experiment Design

As a leanCoP-style baseline we use leanCoP^C[7]. This is a Python version of the leanCoP Prolog implementation, with the two recommended predicates used in the first step of the schedule by Otten[7, § 5.5].

`cut` Choose one extension or reduction and commit to it, similarly to Prolog's `!` operator.

`comp(7)` The prover increases the depth bound by 1 whenever the current tree saturates, until reaching maximum height 7. Afterwards, the height limit is removed.

These simple policies tend to be very effective in tableau provers[8].

We add Python implementations in the connection prover of the two versions of SATCoP presented by Rawson and Reger[13]: SATCoP$_0{}^C$, corresponding to SATCoP$_0$, and SATCoP^C, corresponding to the version of SATCoP with the improvements in the "Future Research" section that was submitted to CASC. These implementations are faithful to the originals, and they allow a better comparison with SATResetCoP.

An overview of these strategies is shown in Table 1.

Each problem-strategy pair is run for at most 120 seconds and 1 million steps, on a single core of the University of Cambridge CSD3 Cluster with dual-socket Intel Xeon Platinum 8368Q CPUs at 2.60 GHz, with 38 cores per socket and one thread per core, and 3370 MiB RAM per core[19]. We thank the Computer Science Lab of the University of Cambridge for providing this computer.

4.2 Results

4.2.1 Individual Results

SATResetCoP considerably increases the number of problems solved by both SATCoP$_0$^{C} and SATCoP^C under the 120-second time limit, as shown in Table 2. The main driver is that the SAT solver reaches unsatisfiability in less time and fewer steps than in the previous strategies, as seen in Figures 3a and 3b. For the SAT-based variants, the 1 000 000 step limit is rarely reached, and most unsolved problems hit the 120 second limit instead.

The improvement is visible in both problem formats. SATResetCoP solves 462 more problems than SATCoP^C, with 206 additional CNF problems and 256 additional FOF problems. Compared with SATCoP$_0$^{C}, it solves 1100 more problems, an increase of 58.8%, and compared with leanCoP^C it solves 1048 more problems, an increase of 54.5%.

	CNF		FOF		Total	
strategy	Number	%	Number	%	Number	%
leanCoP^C	723	9.3	1200	13.7	1923	11.6
SATCoP$_0$^{C}	804	10.3	1067	12.2	1871	11.3
SATCoP^C	1067	13.7	1442	16.4	2509	15.1
SATResetCoP	1273	16.3	1698	19.3	2971	17.9

Table 2: Number and percentage of CNF and FOF problems solved within the time limit by each strategy. Each strategy component (other than the original SATCoP$_0$^{C}) improves results on both CNF and FOF problems.

Appendix A shows the solution to the TPTP problem SYN052-1[11] in three different strategies. This is an example of the kind of problem on which SATResetCoP performs best, since it can be solved with a relatively small UNSAT core.

4.2.2 Comparative Results

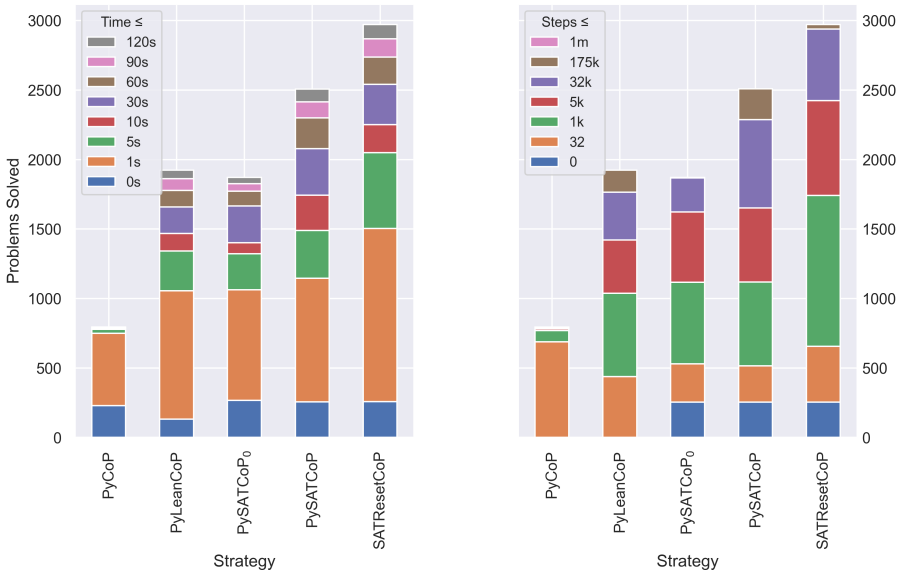
The SAT guidance in SATResetCoP allows it to solve several problems left unsolved by strategies without the same mechanism for accumulating ground clauses, and even by SATCoP$_0$^{C} and SATCoP^C, which use less aggressive guidance. A comparison of newly solved and newly unsolved problems can be seen in Table 3.

Strategy	Newly Solved	Newly Unsolved	Total Solved
leanCoP^C	—	—	1923
SATCoP$_0$^{C}	327	379	1871
SATCoP^C	661	23	2509
SATResetCoP	598	136	2971

Table 3: Problems newly solved and newly unsolved by each strategy compared with the previous strategy in the table, together with the total number of solved problems. This is consistent with previous data that shows SATResetCoP solving hard problems faster. The 136 problems solved by SATCoP^C but not by SATResetCoP correspond mostly to propositional cases where the SAT abstraction does not become contradictory quickly, so aggressive resetting interrupts the tableau branches that can be closed by SATCoP^C.

Figure 4 shows the number of problems solved by one strategy but not by another.

The improvement between SATResetCoP and the best previous experimental strategy, our SATCoP^C implementation of SATCoP, is especially visible in TPTP categories where propositional contradictions are reached quickly in the accumulated ground abstraction, such as SYN, SWC,



(a) Problems solved, stacked by time in seconds.

(b) Problems solved, stacked by number of steps. SAT-based strategies can solve a problem in 0 steps if the initial ground SAT clause set is already unsatisfiable.

Figure 3: Results of running the strategies on all problems under limits of 120 seconds and 1 million steps. SATResetCoP solves more problems in fewer steps than in previous experiments, and also faster except for the niche category of $\leq 1s$ solves.

Fredrik: Remove PyCoP from figure. Rename the Python baselines from Py-X to X^C future-proofs the paper if Connections eventually moves to another language, and emphasises that the common Connections backend is what matters, not Python.

and SWV. As expected, this strategy also solves problems in fewer steps, as can be seen in Figure 5.

Strategy	SYN	SWC	SWV	GEO	NUM	Rest	Total
leanCoP^C	486	14	163	216	86	958	1923
SATCoP$_0{}^C$	459	2	161	177	90	982	1871
SATCoP^C	549	11	173	260	151	1365	2509
SATResetCoP	672	64	215	293	179	1548	2971

Table 4: Number of problems solved, per category, for each strategy.

4.2.3 Soundness Verification: Problem Satisfiability

The TPTP library includes declared statuses for its problems[18]. While this does not demonstrate soundness, it provides a useful check that the strategy has not become unsound by mistake.

We make a second run of SATResetCoP with a longer 10-minute limit, whose results can be seen in Table 5. The results are consistent with what we would expect if the prover were

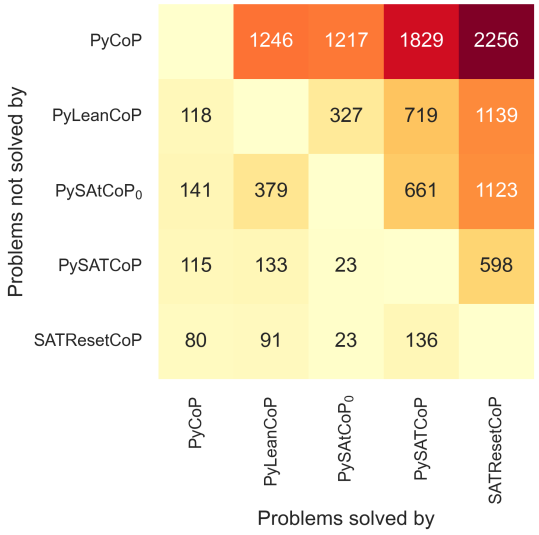


Figure 4: Pairwise comparison of strategy results: the cell in column y and row x shows the number of problems solved by strategy y, but not by strategy x. For example, SATResetCoP solved 1123 TPTP problems that $\mathsf{SATCoP}_0{}^{\mathsf{C}}$ did not.

Fredrik: Remove PyCoP from figure. Rename the Python baselines from Py-X to X^{C} future-proofs the paper if Connections eventually moves to another language, and emphasises that the common Connections backend is what matters, not Python.

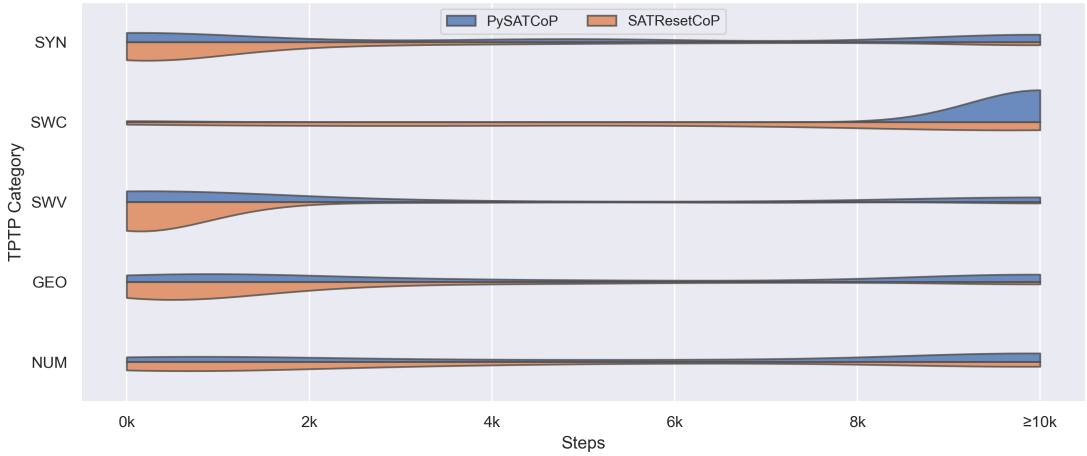


Figure 5: Density of problems solved at each step count by $\mathsf{SATCoP}^{\mathsf{C}}$ and SATResetCoP on the TPTP categories SYN, SWC, SWV, GEO, and NUM. The area of each violinplot is the same; besides solving more problems in each category, SATResetCoP solves many problems in fewer steps. Unsolved problems are collected in the $\geq 10k$ category on the right.

sound, as it does not find unsatisfiability ("Solved") in any problem marked satisfiable or countersatisfiable. It also does not solve any unknown or open problems; while this limits the

number of new discoveries, any such solutions would need to be checked closely to ensure that they were not caused by an incorrect prover.

	FOF		CNF	
Declared status	Unsolved	Solved	Unsolved	Solved
Satisfiable	350	0	1094	0
CounterSatisfiable	732	0	0	0
Unknown	94	0	745	0
Open	4	0	28	0
Unsatisfiable	100	72	4496	1433
ContradictoryAxioms	204	39	0	0
Theorem	4426	1823	0	0

Table 5: Number of problems solved and unsolved by SATResetCoP, grouped by TPTP declared status. No satisfiable or countersatisfiable problem was incorrectly proven to be unsatisfiable.

5 Conclusion

The experiments on SATResetCoP show that the extreme of *backtracking on backtracking*, which is preventing backtracking at all and resetting the search trees instead, is a valid way to improve tableau provers.

Section 4.2.2 shows that, on a 1 million step and 120 second time limit, SATResetCoP solves 2971 problems. This is 462 more problems than SATCoP^{C}, an 18% increase, and 1048 more problems than leanCoP^{C}, a 54% increase. Pairwise, SATResetCoP solves 598 problems that SATCoP^{C} does not, and 1139 problems that leanCoP^{C} does not.

The improvement is not uniform across TPTP. It is strongest in categories where useful ground clauses accumulate quickly: SATResetCoP solves 672 SYN problems, 64 SWC problems, and 215 SWV problems, compared with 549, 11, and 173 for SATCoP^{C}. It also improves on GEO and NUM, solving 293 and 179 problems respectively.

The newly unsolved problems show the other side of this tradeoff. SATResetCoP loses 136 problems solved by SATCoP^{C}, mostly in set theory, number theory, logic calculi, and commonsense reasoning. These are cases where the SAT abstraction does not become contradictory quickly enough, and the aggressive reset policy can discard tableau branches that a less aggressive prover eventually closes.

This suggests that accumulating ground clauses is a useful objective for connection tableaux, but not a replacement for all tableau control. The most promising direction is therefore not a single fixed reset policy, but a schedule or controller that can use aggressive resets when ground instances are informative and fall back to ordinary connection search when they are not.

6 Future Work

6.1 Schedule with different provers

SATResetCoP still struggles with a significant number of problems, as seen in Table 3. It would be worth experimenting with adding this strategy to a schedule, following the examples of Otten and Mangla[7, 6], and possibly running a different strategy or solver if no solution is found after a fixed amount of time or number of steps.

6.2 Further Experiments

The current implementation uses CaDiCaL as its incremental SAT solver. Future experiments should test whether the results are stable under different SAT solvers, solver restart policies, and incremental interfaces. This matters because SATResetCoP relies on the solver not only for the final unsatisfiability result, but also for the model information used to guide tableau extensions.

It would also be useful to vary the reset policy itself. The present policy resets aggressively when the current tableau stops producing new ground clauses, but the newly unsolved problems suggest that some domains need longer tableau branches before the useful ground instances appear. A broader experiment could compare fixed reset budgets, category-specific schedules, and policies that delay resets when recent SAT models still give useful literal guidance.

6.3 CASC Submission

SATResetCoP will also be packaged as a CASC entrant. This will test whether the improvements seen here survive the standard competition setting, including the official hardware, time limits, problem selection, and input requirements. It will also make the comparison with earlier connection-tableau systems more direct, since both leanCoP and SATCoP have been evaluated through CASC.

Preparing a CASC version requires more than submitting the current research prototype. The prover should expose a stable command-line interface, handle TPTP input robustly, report SZS statuses consistently, and include a reproducible installation path for the SAT backend. This would turn the current experimental result into a system that can be compared against the broader ATP ecosystem.

7 Acknowledgements

This work was performed using resources provided by the Cambridge Service for Data Driven Discovery (CSD3) operated by the University of Cambridge Research Computing Service (www.csd3.cam.ac.uk), provided by Dell EMC and Intel using Tier-2 funding from the Engineering and Physical Sciences Research Council (capital grant EP/T022159/1), and DiRAC funding from the Science and Technology Facilities Council (www.dirac.ac.uk).

Fredrik: I would drop the appendix and move code instructions to README in repo

A Solving example

B Usage Example

The repo in https://github.com/mfixman/resetsatcop contains two Python files that can be run standalone.

`pycop.py` gets as input a single strategy and a single `.p` file, in either FOF or CNF format. This can be expressed as the path to this file, or a TPTP file specification; in the latter case, the file's path is expected to be inside where the main TPTP directory, which can be specified either with the `-tptp` option or with the `$TPTP` environment variable points.

```
$ python pycop.py --strategy SATResetCoP --print-trace --debug PUZ020-1.p
Start st1: [person(wife)]
Extension ex3: person(wife) -> [-person(X23), -a_truth(statement_by(X23)), knight(X23)] [cut]
Reset 1
Start st4: [-person(X29), knight(X29), knave(X29)]
Extension ex1: -person(X29) -> [person(wife)] [cut]
Extension ex1: knight(X29) -> [-says(X47, Y47), -knight(X47), a_truth(Y47)] [cut]
Reset 1
Start st10: [-a_truth(statement_by(X62)), -person(X62), knight(X62)]
Extension ex5: -a_truth(statement_by(X62)) -> [a_truth(statement_by(husband)), -knight(wife)] [cut]
Start st15: [-knight(husband), knight(wife), -a_truth(statement_by(husband))]
Extension ex1: -knight(husband) -> [-person(X78), knight(X78), -a_truth(statement_by(X78))] [cut]
Extension ex1: knight(wife) -> [a_truth(Y82), -knight(X82), -says(X82, Y82)] [cut]
Reset 1
{'info': 'Theorem (SAT); (-person(husband), -a_truth(statement_by(husband)), knight(husband));
     (a_truth(statement_by(husband)), -knight(wife)); (a_truth(statement_by(husband)), knight(wife));
     (-knight(husband)); (person(husband))', 'steps': 10}
```

`compare_strategies.py` can compare the result of running different strategies in a group of files. The program takes a series of strategies and a directory or set of `.p` problems as an argument (all TPTP problems by default), and returns a CSV with solving information for each strategy/problem pair.

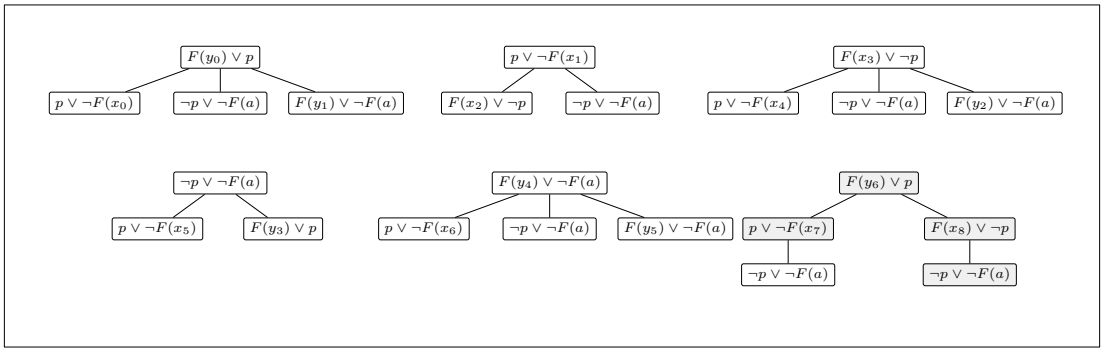
The program aggressively uses multiprocessing, and will try to "fit" each strategy/problem pair into a single processor core. Additionally, it keep partial information of solved problems; runs of `compare_strategies.py -name A` after the first one will retry from where the previous run ended, even if that run crashed or was killed for some reason.

```
$ python compare_strategies.py --name LongTest --strategies SATCoP SATResetCoP $TPTP/Problems/PUZ
Run: partial=partial/LongTest_PUZ  csv=results/LongTest/PUZ.csv  tasks=262  completed=0  remaining=262
     workers=20  timeout=120s
Problem      Strategy     Result
--------------------------------
PUZ004-1.p   SATCoP       0 / 0ms
PUZ004-1.p   SATResetCoP  0 / 0ms
PUZ001-1.p   SATCoP       81 / 17ms
PUZ001-1.p   SATResetCoP  66 / 16ms
[...]

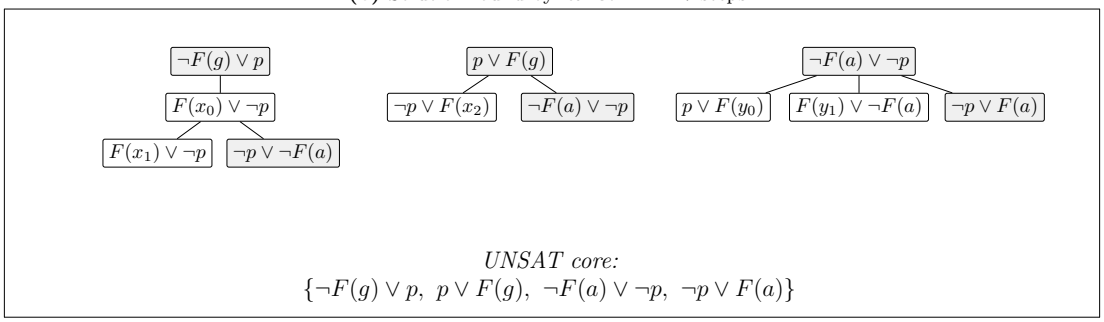
$ head results/LongTest/PUZ.csv | cut -d',' -f1-7 | column -t -s',' -R6,7
problem     strategy     status   result         solved   steps  time_ms
PUZ001+1.p  SATCoP       solved   Theorem (SAT)  True      2076      778
PUZ001+1.p  SATResetCoP  solved   Theorem (SAT)  True       989      821
PUZ001+2.p  SATCoP       timeout                 False    81106   120000
PUZ001+2.p  SATResetCoP  solved   Theorem (SAT)  True     35854    77695
PUZ001-1.p  SATCoP       solved   Theorem (SAT)  True        81       15
```

$$p \vee \neg F(X) \qquad F(X) \vee \neg p$$
$$F(Y) \vee p \qquad \neg p \vee \neg F(a) \qquad F(Y) \vee \neg F(a)$$

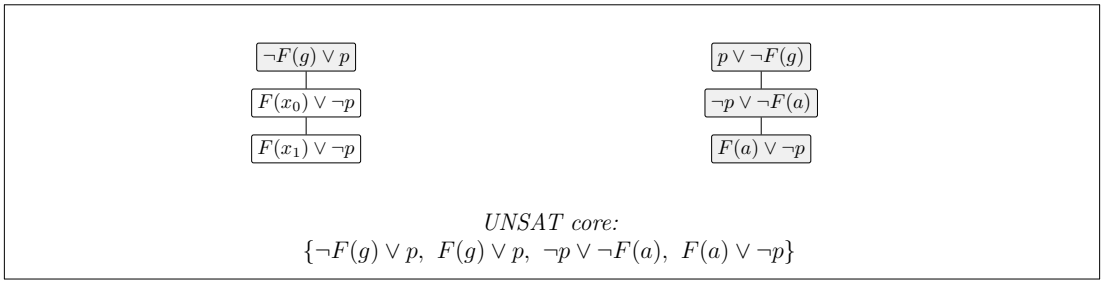
(a) Problem SYN052-1.p[11], adapted from https://tptp.org/TPTP/Problems/SYN/SYN052-1.p.



(b) Solution found by leanCoP^{C} in 27 steps.



(c) Solution found by SATCoP^{C} in 19 steps.



(d) Solution found by SATResetCoP in 9 steps.

Figure 6: Unsatisfiability proofs for the TPTP problem SYN052-1[11] produced by three provers: leanCoP^{C}, SATCoP^{C}, and SATResetCoP.

```
PUZ001-1.p  SATResetCoP  solved   Theorem (SAT)  True        66       15
PUZ001-2.p  SATCoP       timeout                 False   497938   120000
PUZ001-2.p  SATResetCoP  timeout                 False   513586   120000
PUZ001-3.p  SATCoP       timeout                 False   245563   120000
```

References

[1] Leo Bachmair, Harald Ganzinger, David McAllester, and Christopher Lynch. Resolution Theorem Proving. In *Handbook of Automated Reasoning*, pages 19–99. Elsevier, 2001.

[2] Armin Biere, Tobias Faller, Katalin Fazekas, Mathias Fleury, Nils Froleyks, and Florian Pollitt. CaDiCaL 2.0. In *Computer Aided Verification*, volume 14681 of *Lecture Notes in Computer Science*, pages 133–152. Springer, 2024.

[3] Free Software Foundation. GNU lesser general public license, version 3, 2007.

[4] Michael Färber. A Curiously Effective Backtracking Strategy for Connection Tableaux, 2024.

[5] Reinhold Letz and Gernot Stenz. Model Elimination and Connection Tableau Procedures. In *Handbook of Automated Reasoning*, pages 2015–2114. Elsevier, 2001.

[6] C. Mangla, S.B. Holden, and L.C. Paulson. Bayesian ranking for strategy scheduling in automated theorem provers. In *International Joint Conference on Automated Reasoning (IJCAR)*, pages 559–577„ 2022.

[7] Jens Otten. Restricting backtracking in connection calculi. *AI Communications*, 23(2–3):159–182, 2010.

[8] Jens Otten. 20 years of leanCoP — an overview of the provers. In *Proceedings of the 1st International Workshop on Automated Reasoning with Connection Calculi (AReCCa@TABLEAUX 2023)*, volume 3613 of *CEUR Workshop Proceedings*, 2023.

[9] Jens Otten and Wolfgang Bibel. Advances in Connection-Based Automated Theorem Proving. In Mike Hinchey, Jonathan P. Bowen, and Ernst-Rüdiger Olderog, editors, *Provably Correct Systems*, pages 211–241. Springer International Publishing, 2017.

[10] Jens Otten and Sean Holden. A Syntax for Connection Proofs, 2023.

[11] Francis Jeffry Pelletier. Seventy-five problems for testing automatic theorem provers. *Journal of Automated Reasoning*, 2(2):191–216, 1986.

[12] Michael Rawson, Clemens Eisenhofer, and Laura Kovács. Constraint Learning for Non-confluent Proof Search. In Gian Luca Pozzato and Tarmo Uustalu, editors, *Automated Reasoning with Analytic Tableaux and Related Methods*, volume 15980, pages 103–119. Springer Nature Switzerland, 2026.

[13] Michael Rawson and Giles Reger. Eliminating Models During Model Elimination. In Anupam Das and Sara Negri, editors, *Automated Reasoning with Analytic Tableaux and Related Methods*, volume 12842, pages 250–265. Springer International Publishing, 2021.

[14] Alexandre Riazanov and Andrei Voronkov. The design and implementation of VAMPIRE. *AI Communications*, 15(2–3):91–110, 2002.

[15] Fredrik Rømming, Jens Otten, and Sean B. Holden. Connections: Markov Decision Processes for Classical, Intuitionistic and Modal Connection Calculi. In *Proceedings of the 1st International Workshop on Automated Reasoning with Connection Calculi*, volume 3613 of *CEUR Workshop Proceedings*, pages 107–118, 2023.

[16] Stephan Schulz. Light-weight integration of SAT solving into first-order reasoners: First experiments. In Laura Kovács and Andrei Voronkov, editors, *Vampire 2017. Proceedings of the 4th Vampire Workshop*, volume 53 of *EPiC Series in Computing*, pages 9–19. EasyChair, 2018.

[17] Stephan Schulz and Martin Möhrmann. Performance of Clause Selection Heuristics for Saturation-Based Theorem Proving. In Nicola Olivetti and Ashish Tiwari, editors, *Automated Reasoning*, volume 9706, pages 330–345. Springer International Publishing, 2016.

[18] Geoff Sutcliffe. The TPTP problem library and associated infrastructure: From CNF to TH0, TPTP v6.4.0. *Journal of Automated Reasoning*, 59(4):483–502, 2017.

[19] University of Cambridge Research Computing Services. Ice Lake Nodes. https://docs.hpc.cam.ac.uk/hpc/user-guide/icelake.html, 2026. Accessed 13 May 2026.